

Persistência Local com SQLite

Update, Delete, List e Migrations

Thales Lagemann

O que vimos anteriormente

- Conceito de banco local com SQLite.
- Configuração do pacote e abertura do banco.
- Criação da tabela inicial.
- Inserção de registros com operações básicas de *insert*.
- Classes de Modelos e DAOs.

Quando usar update

- Editar nome, e-mail, curso ou outros dados do aluno.
- Corrigir cadastro preenchido incorretamente.
- Atualizar informações acadêmicas sem recriar o registro.
- Evitar duplicidade quando o objetivo é alterar um aluno já existente.

Exemplo de update

```
Future<int> updateAluno(Aluno aluno) async {  
  final db = await database;  
  
  return await db.update(  
    'alunos',  
    aluno.toMap(),  
    where: 'id = ?',  
    whereArgs: [aluno.id],  
  );  
}
```

- Sempre usar `where` com `whereArgs`.
- Atualizar apenas os campos necessários.
- Retornar a quantidade de linhas afetadas para confirmar sucesso.

- Remover cadastro criado por engano.
- Excluir registros de uma base de teste durante desenvolvimento.
- Limpar registros desatualizados quando houver regra para isso.

Exemplo de delete

```
Future<int> deleteAluno(int id) async {  
  final db = await database;  
  
  return await db.delete(  
    'alunos',  
    where: 'id = ?',  
    whereArgs: [id],  
  );  
}
```

- Nunca executar `delete` sem especificar `where`, pois isso pode remover todos os registros da tabela.
- Confirmar a ação na interface quando fizer sentido.
- Tratar impactos quando houver relacionamento entre tabelas, pois a remoção de um registro pode deixar dados órfãos ou inconsistentes em outras tabelas.
- Em alguns casos pode fazer mais sentido `inativar` do que deletar.

Exemplo de consulta e mapeamento

```
Future<List<Aluno>> getAlunos() async {  
  final db = await database;  
  final result = await db.query(  
    'alunos',  
    orderBy: 'nome ASC',  
  );  
  
  return result.map((e) => Aluno.fromMap(e)).toList();  
}
```

- Usar `orderBy` para ordenação quando fizer sentido.
- Aplicar filtros por status, texto ou categoria.
- Centralizar o mapeamento em `fromMap()`.
- Recarregar dados após *insert*, *update* e *delete*.

- Mudanças controladas na estrutura do banco ao longo do tempo.
- Permitem evoluir o schema sem perder compatibilidade.
- São acionadas quando a versão do banco muda.
- Evitam quebrar registros que já possuem dados salvos.
- Úteis para atualizar programas que já estão em produção.

Exemplos de mudanças comuns

- Adicionar uma nova coluna.
- Criar uma nova tabela.
- Renomear campos via estratégia de cópia de dados.
- Popular valores padrão para registros antigos.

Exemplo com onUpgrade

```
openDatabase(  
    'escola.db',  
    version: 2,  
    onCreate: (db, version) async {  
        await db.execute('CREATE TABLE alunos (  
            'id INTEGER PRIMARY KEY AUTOINCREMENT, '  
            'nome TEXT, email TEXT, curso TEXT)');  
    },  
    onUpgrade: (db, oldVersion, newVersion) async {  
        if (oldVersion < 2) {  
            await db.execute(  
                'ALTER TABLE alunos ADD COLUMN telefone TEXT');  
        }  
    },  
);
```

- Versionar o banco com clareza.
- Criar migrations pequenas e previsíveis.
- Testar atualização saindo de versões antigas.

- Alterar schema manualmente sem subir a versão.
- Assumir que todos os usuários estão na versão anterior imediata.
- Misturar lógica de negócio com lógica de migration.
- Não prever valores padrão para colunas novas.

- **Update:** altera registros existentes com segurança.
- **Delete:** remove dados com critério e cuidado.
- **List:** consulta dados e abastece a interface.
- **Migrations:** mantêm o banco evoluindo sem quebrar o app.

Obrigado pela atenção!

Dúvidas ou sugestões:
`talagemann@inf.ufsm.br`